💪 Supervised ML (Continued)

Computational Communication Science II

May 11, 2026

📚 Agenda

- Recap of Week 4
- Validation concepts
- Validation metrics

🔁 Week 4 Recap

What is Machine Learning?

Instead of writing rules by hand, we **teach the computer with examples**.

> Show it thousands of tweets labeled "toxic" or "not toxic"
> → it learns the pattern
> → it labels **new** tweets on its own

Learn from examples → Generalize to new data

Three Families of ML

Family	Core Idea	Example
Supervised	Learn from 'labeled' examples	Toxic tweet detector
Unsupervised	Find hidden patterns, no labels	Group similar news articles
Reinforcement	Learn by trial & error	Train a chatbot

👉 We focus on Supervised ML

Supervised ML in 3 Steps

1. **Features / X** - the inputs we give the model
   (e.g., number of insult words, % CAPS LOCK, tweet length)

2. **Label / y** - the correct answer we want to predict
   (e.g., Toxic 😡 or Not Toxic 🙂)

3. **Model / f** - the algorithm that learns the pattern
   (train it once → use it to predict new cases)

Regression vs. Classification

Regression

Predict a **number**

> What is the toxicity **score** of this tweet?
> → *2.1, 7.8, 9.3...*

Classification

Predict a **category**

> Is this tweet toxic?
> → *Yes (1) or No (0)...*
>
> What type of toxicity?
> → *Harassment, Hate speech...*

Models We Covered

Model	Best for
Linear Regression	Predicting a number
Logistic Regression	Simplest yes/no
k-NN	Small datasets
Decision Tree	Explainable rules
Random Forest	High accuracy
SVM	Many features
Neural Network	Large, complex data

Naive Bayes Algorithm

- A type of supervised ML that makes predictions using probability (how likely something is to happen)
- Example: If it's **cloudy**, then there is a **70%** chance of rain
- Analogy: Like guessing if a fruit is an apple or orange based on its color and shape
- Works well for text classification

How Does Naive Bayes Work?

Imagine we are predicting if an email is spam or not spam based on it having words like "free" or "offer".

- Step 1: Learn from examples:
  - Look at emails we already know are spam or not spam.
  - Count how often words like "free" or "offer" appear in spam vs. not spam.
- Step 2: For a new email, check its words.
- Step 3: Calculate which is more likely: spam or not spam.

The "Naive" Part

- Naive Bayes assumes all clues (like words in an email) are independent.
- Example: It assumes "free" and "offer" don’t affect each other’s meaning.
- This is "naive" because, in reality, words can be related.
- Why it’s okay: It’s simple and still works well for many problems!

A Simple Example

Let's classify a fruit as Apple or Orange:

Fruit	Color	Shape	Taste
Apple	Red	Round	Sweet
Orange	Orange	Round	Tangy

New fruit: It is "Red, Round, Sweet", what is it?

Naive Bayes checks:

- How often is "Red" in Apples vs. Oranges? **95% or 0.95**
- How often is "Round" in Apples vs. Oranges? **50% or 0.50**
- How often is "Sweet" in Apples vs. Oranges? **75% or 0.75**

Combines these probabilities to guess $\rightarrow$ Apple.

Why split data into *train* and *test* sets?

A key challenge in SML is ensuring the model *generalizes* to new, unseen data.

What is a Train/Test split?

Training Set:

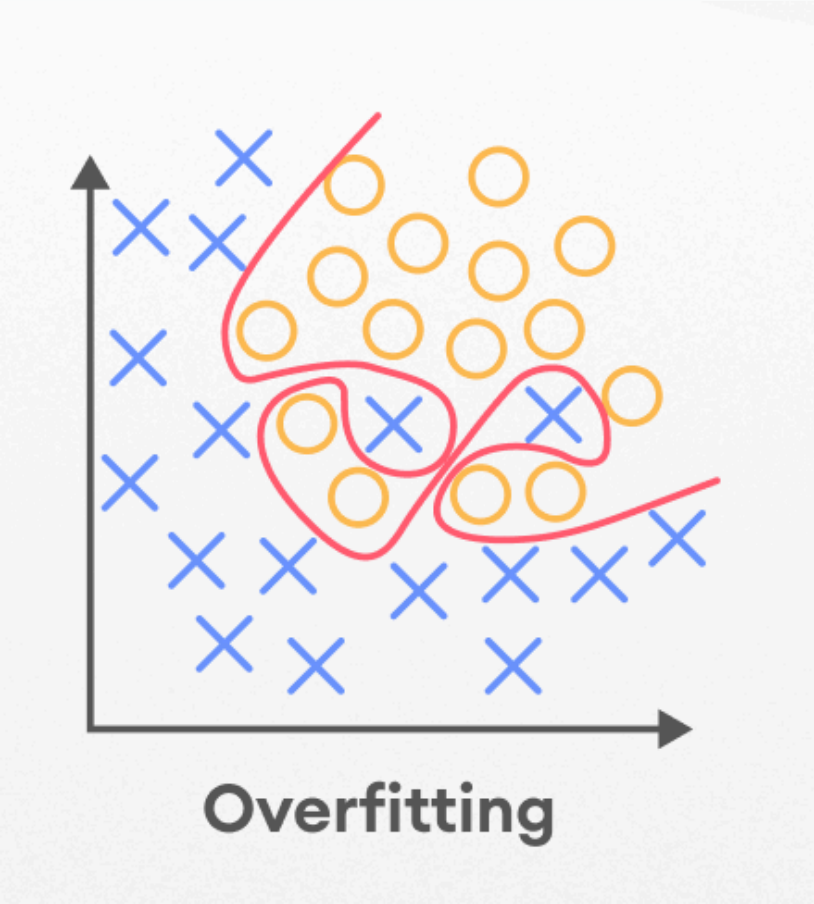
- The portion of data used to **train (or fit)** the model.
- The model "learns" from this data.

Test Set:

- A separate portion of data kept **untouched** during training.
- Used to **evaluate** the model's performance on unseen data.

Note: The Test Set acts as a placeholder for UNSEEN real-world data

What if we use 100% data for training?



We run into *overfitting*, which means:

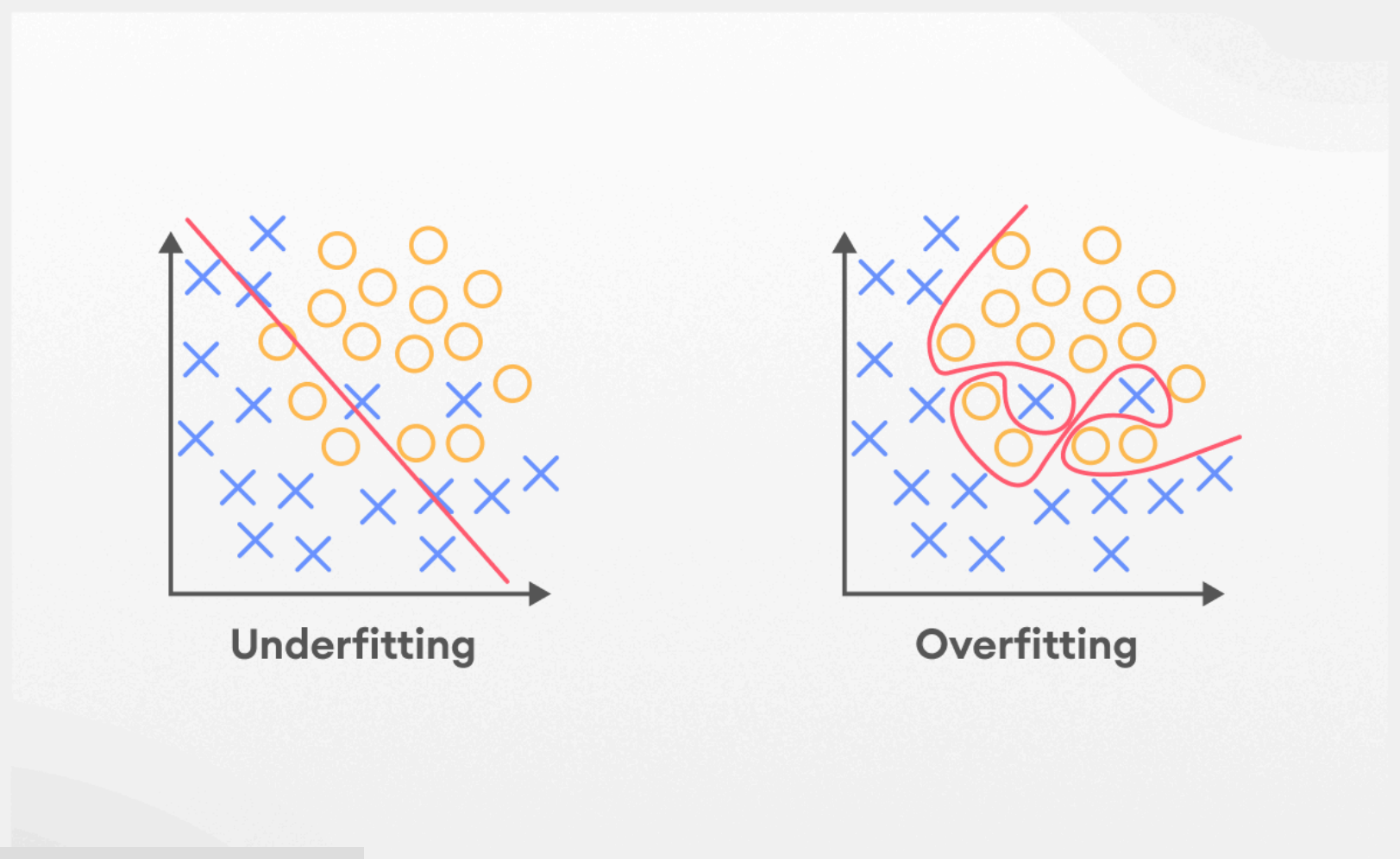
- The model performs very well on the training set 🙂
- But...it fails to predict new data accurately 🤮
- Our model will just memorize the current data, and not learn useful patterns (that help handle fresh data)

📚 A Simple Analogy

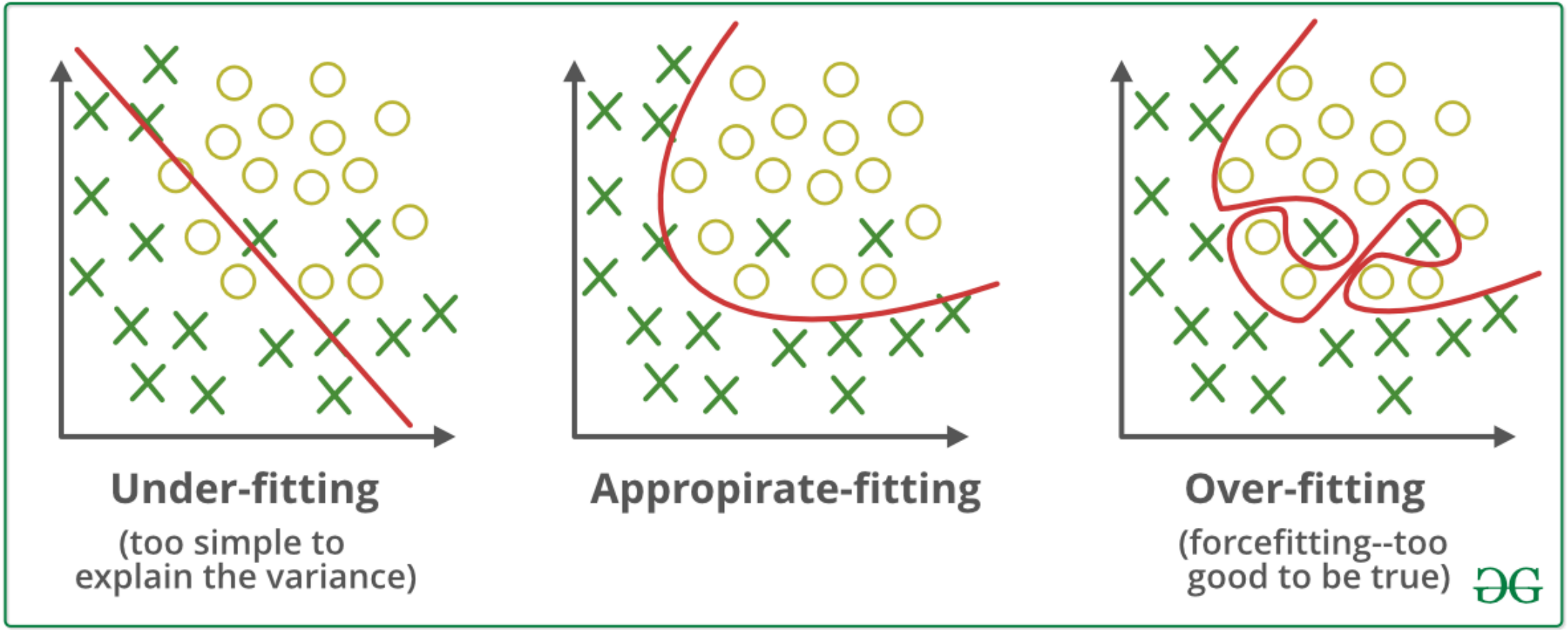
- Imagine you're studying for an exam:
  - **Training:** You practice a lot and memorize all exercise questions
  - **Testing:** You take an exam containing completely new questions
- Memorizing might help you ace the existing test, but true understanding is shown in the new exam
- The **train/test split** ensures that the model isn't just memorizing, but actually learning patterns that apply to new data

What if we use 1% data for training?

Underfitting can be a problem too 😖



⚖️ Balance is important



Why We Need Train/Test Splits

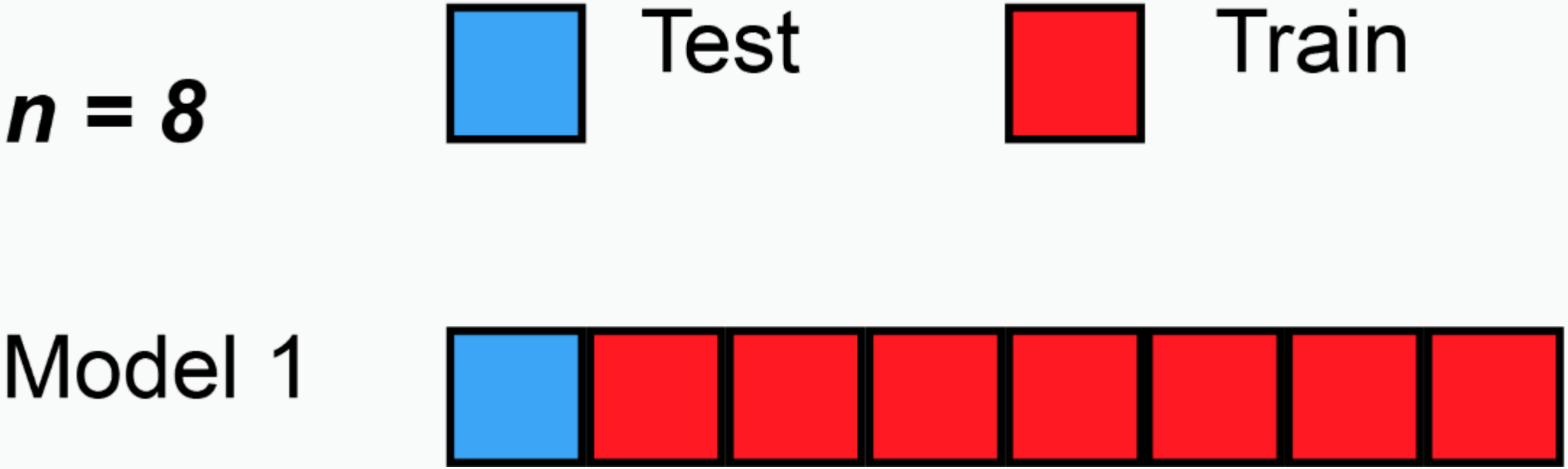
- **Avoiding Overfitting:** If we evaluate a model on the same data we trained it on, we might overestimate its performance.
- **Measuring Generalization:** The test set gives us a realistic estimate of how our model will perform on new data.
- **Reliable Performance Metrics:** Metrics such as accuracy, precision, or recall computed on the test set better reflect true predictive power.

In Summary

- **Train/Test Splits** are crucial for ensuring your model generalizes well to unseen data.
- They help prevent overfitting by keeping test data separate from training.
- Always assess your model's performance on the test set to get a realistic idea of its real-world capabilities.

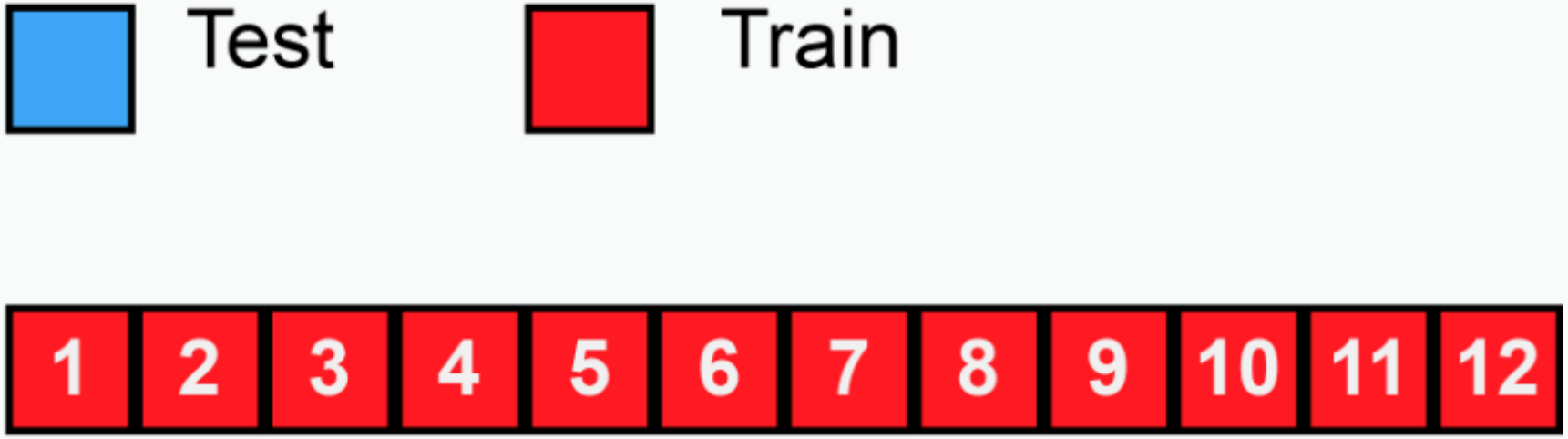
A Better Way → Cross Validation

***n* = 8**



An Even Better Way → *k*-fold Cross Validation

***n* = 12**
***k* = 3**



⏰

Confusion Matrices

A Confusion Matrix

	Actual: Dog	Actual: Not Dog
Predicted: Dog		
Predicted: Not Dog		

A Confusion Matrix

	Actual: Dog	Actual: Not Dog
Predicted: Dog	?	?
Predicted: Not Dog	?	?

A Confusion Matrix

	Actual: Dog	Actual: Not Dog
Predicted: Dog	True Positive (TP)	False Positive (FP)
Predicted: Not Dog	False Negative (FN)	True Negative (TN)

A Confusion Matrix

	Actual: Dog	Actual: Not Dog
Predicted: Dog	True Positive (TP)	False Positive (FP)
Predicted: Not Dog	False Negative (FN)	True Negative (TN)

True Positive (TP): It is the total counts having both predicted and actual values are Dog.

True Negative (TN): It is the total counts having both predicted and actual values are Not Dog.

False Positive (FP): It is the total counts having prediction as Dog while actually Not Dog.

False Negative (FN): It is the total counts having prediction as Not Dog while actually, it is Dog.

Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result										

Fill in the last row as TP, FP, FN, or TN.

Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

Actual	Dog	Dog	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Not Dog	Not Dog
Predicted	Dog	Not Dog	Dog	Not Dog	Dog	Dog	Dog	Dog	Not Dog	Not Dog
Result	TP	FN	TP	TN	TP	FP	TP	TP	TN	TN

True Positive Counts =

False Positive Counts =

True Negative Counts =

False Negative Counts =

	Actual: Dog	Actual: Not Dog
Predicted: Dog	True Positive (TP = 5)	False Positive (FP = 1)
Predicted: Not Dog	False Negative (FN = 1)	True Negative (TN = 3)

Accuracy?

What proportion of predictions are correct predictions?

$$\frac{TP+TN}{TP+TN+FP+FN}$$

	Actual: Dog	Actual: Not Dog
Predicted: Dog	True Positive (TP = 5)	False Positive (FP = 1)
Predicted: Not Dog	False Negative (FN = 1)	True Negative (TN = 3)

Precision?

What proportion of the images labeled as dogs were actually dogs?

$$\frac{TP}{TP+FP}$$

	Actual: Dog	Actual: Not Dog
Predicted: Dog	True Positive (TP = 5)	False Positive (FP = 1)
Predicted: Not Dog	False Negative (FN = 1)	True Negative (TN = 3)

Recall?

What proportion of the actual dog images were correctly identified as dogs?

$$\frac{TP}{TP+FN}$$

	Actual: Dog	Actual: Not Dog
Predicted: Dog	True Positive (TP = 5)	False Positive (FP = 1)
Predicted: Not Dog	False Negative (FN = 1)	True Negative (TN = 3)

F1-score?

$$\frac{2 \times Precision \times Recall}{Precision + Recall}$$

Aside: Connection with hypothesis testing

Type I error: Null hypothesis is true, but we reject it

Type II error: Null hypothesis is false, but we fail to reject it

What Metric to Use When?

Accuracy: Use when groups are roughly balanced and every prediction is equally important. *Example: Predicting if a student will pass or fail an exam when pass/fail cases are similar in number.*

Precision: Use when you want to minimize false positives (i.e., you don't want to incorrectly predict a positive). *Example: In spam detection, you don't want to classify legitimate emails as spam (high precision).*

Recall: Use when you want to minimize false negatives (i.e., you don't want to miss any positive cases). *Example: In disease detection, you want to catch as many real cases as possible (high recall).*

F1 Score: Use when you want a balance between precision and recall, especially on **imbalanced** datasets. *Example: Fraud detection: you care both about catching fraud (recall) and avoiding false alarms (precision).*

Next week

Week 8: Coding in an Academic Context

Lecture: Monday, May 18, 2025

Readings:

- Bender et al. (2021): On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?
- Hube et al. (2019): Understanding and Mitigating Worker Biases in the Crowdsourced Collection of Subjective Judgments

Practice

Open your jupyter notebooks

📥 Practice 1: Load the Data

Use `pandas` to read `StreamingMediaReviews.csv`.
Then pull the two columns out as lists `reviews` and `labels`.

```
import pandas as pd

df = pd.read_csv("StreamingMediaReviews.csv")
print(df.head())

reviews = ...   # ← which column?
labels  = ...   # ← which column?

print(reviews[:3])
print(labels[:3])
print("Lengths match:", len(reviews) == len(labels))
```

✅ Solution 1: Load the Data

```
import pandas as pd
from collections import Counter
import matplotlib.pyplot as plt

df = pd.read_csv("StreamingMediaReviews.csv")

reviews = df['review'].tolist()
labels  = df['label'].tolist()

print(reviews[:3])
print(labels[:3])
print("Lengths match:", len(reviews) == len(labels))

counts = Counter(labels)
plt.bar(counts.keys(), counts.values())
plt.title("Label Distribution")
plt.show()
```

✂️ Practice 2: Train / Test Split

Split `reviews` and `labels` into **80% train** and **20% test**.
Why is `random_state` important?

```
from sklearn.model_selection import train_test_split

reviews_train, reviews_test, y_train, y_test = train_test_split(
    ???, ???, test_size=???, random_state=???
)
print("Train:", len(reviews_train), "| Test:", len(reviews_test))
```

✅ Solution 2: Train / Test Split

```
from sklearn.model_selection import train_test_split

reviews_train, reviews_test, y_train, y_test = train_test_split(
    reviews, labels, test_size=0.2, random_state=42
)
print("Train:", len(reviews_train), "| Test:", len(reviews_test))
```

> `random_state=42` fixes the random seed so the split is **identical every run** - essential for reproducible research.

📐 Practice 3: Vectorize Text

ML models need **numbers**, not words. `CountVectorizer` converts each review into a vector of word counts.

- Remove common English stop words
- `fit_transform` on **train** only - then `transform` test

```
from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(stop_words=???)
X_train = cv.???(reviews_train)   # fit AND transform
X_test  = cv.???(reviews_test)    # transform ONLY

print(X_train.shape)
```

✅ Solution 3: CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(stop_words="english")
X_train = cv.fit_transform(reviews_train)   # learn vocab + convert
X_test  = cv.transform(reviews_test)        # use same vocab

print(X_train.shape)   # (n_reviews, n_unique_words)
```

> Never `fit_transform` on test data - that leaks information from the test set!

🧠 Practice 4: Train Naive Bayes

Train a `MultinomialNB` model and predict labels for the test set.

```
from sklearn.naive_bayes import MultinomialNB

nb = ???
nb.???(X_train, y_train)      # train
y_pred = nb.???(X_test)       # predict

print(y_pred[:5])
```

✅ Solution 4: Naive Bayes

```
from sklearn.naive_bayes import MultinomialNB

nb = MultinomialNB()
nb.fit(X_train, y_train)      # learn from training data
y_pred = nb.predict(X_test)   # predict labels for test data

print(y_pred[:5])
```

📊 Practice 5: Evaluate the Model

Print a **classification report** comparing actual vs predicted labels.
What do precision, recall, and F1-score each tell you?

```
from sklearn.metrics import classification_report

print(???(y_test, y_pred))
```

✅ Solution 5: Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Metric	Meaning
Precision	Of predicted positives, how many were correct?
Recall	Of all actual positives, how many did we catch?
F1	Balance between precision & recall

🚀 Practice 6: TF-IDF + Logistic Regression

Swap in `TfidfVectorizer` and `LogisticRegression`. Does the report improve?

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

# vectorize → fit model → predict → print classification report
```

✅ Solution 6: TF-IDF + Logistic Regression

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

tfidf = TfidfVectorizer(stop_words="english")
X_train_tf = tfidf.fit_transform(reviews_train)
X_test_tf  = tfidf.transform(reviews_test)

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train_tf, y_train)
y_pred_lr = lr.predict(X_test_tf)

print(classification_report(y_test, y_pred_lr))
```